Improving Code Generation by Dynamic Temperature Sampling

Yuqi Zhu, Jia Li ♂, Ge Li*, YunFei Zhao, Jia Li, Zhi Jin, Hong Mei

Peking University

zhuyuqi@pku.org.com, {lijia.stu,lige,zhaoyunfei, lijiaa, zhijin, meih}@.pku.edu.cn

Abstract

Recently, Large Language Models (LLMs) have shown impressive results in code generation. However, existing decoding strategies are designed for Natural Language (NL) generation, overlooking the differences between NL and programming languages (PL). Due to this oversight, a better decoding strategy for code generation remains an open question. In this paper, we conduct the first systematic study to explore a decoding strategy specialized in code generation. With an analysis of loss distributions of code tokens, we find that code tokens can be divided into two categories: challenging tokens that are difficult to predict and confident tokens that can be easily inferred. Among them, the challenging tokens mainly appear at the beginning of a code block. Inspired by the above findings, we propose a simple yet effective method: Adaptive Temperature (AdapT) sampling, which dynamically adjusts the temperature coefficient when decoding different tokens. We apply a larger temperature when sampling for challenging tokens, allowing LLMs to explore diverse choices. We employ a smaller temperature for confident tokens avoiding the influence of tail randomness noises. We apply AdapT sampling to LLMs with different sizes and conduct evaluations on two popular datasets. Results show that AdapT sampling significantly outperforms state-of-the-art decoding strategy.

Introduction

Code generation aims to automatically generate a program that satisfies a natural language requirement. In recent years, Large Language Models (LLMs) have attracted great attention for their potential for automating coding. Noteworthy models like AlphaCode (Li et al. 2022) and Codex (Chen et al. 2021) have demonstrated their impressive ability to solve unforeseen programming challenges.

LLMs rely on a decoding strategy to generate code. Existing decoding strategies for code generation mainly fall into two categories. The first category is search-based methods, which aim to maximize the probability of the next generated token, including greedy search (Black and E 2012) and beam search (Freitag and Al-Onaizan 2017). Nonetheless, the results generated by these methods lack diversity and are prone to generating empty or repetitive results (Zhang et al. 2021). The second category is sampling-based methods, which randomly sample the next token based on the probability distribution. The state-of-the-art (SOTA) approach,

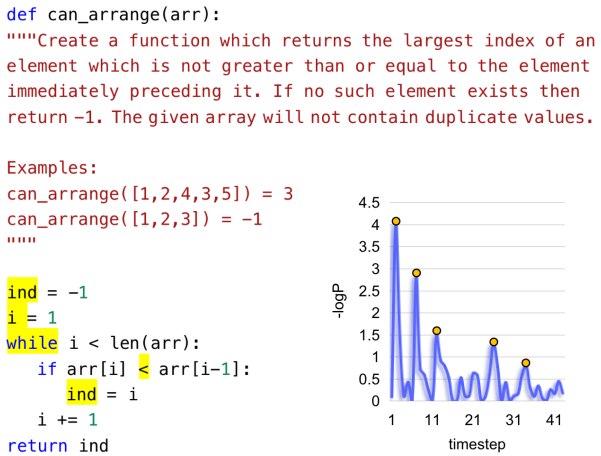

Figure 1: An illustration of a code snippet and its corresponding loss distribution. The challenging tokens are highlighted in yellow.

temperature sampling, reshapes the probability distribution by introducing a temperature coefficient, which serves to control the level of sampling randomness.

In spite of the promising results, temperature sampling has limitations in code generation. Since it is initially used to generate Natural Language (NL) with a flexible syntax, its effectiveness decreases when transitioning to code generation, which is a high-accuracy demanding task. For instance, CodeGeeX (Zheng et al. 2023a), when utilizing temperature sampling, attains 36.6% Pass@15 on the HumanEval dataset. A detailed exploration of this issue is needed to further improve the accuracy of code generation.

In this paper, we present the first systematic study to explore a decoding strategy for code generation. Our contributions can be summarized as follows:

(1) By analyzing the loss distribution of code tokens, we categorize code tokens into challenging tokens and confident tokens. First, we design comparative experiments to investigate the differences in loss distributions between source code and NL text.

Our analysis reveals that the source code suffers lesser variation in loss values during generation than NL text. Next, we visualize the loss distribution of source code. We find an

apparent discrepancy in loss values among different code tokens. In this study, we refer to these tokens with high loss values as challenging tokens. With statistical analysis, we find that challenging tokens frequently appear at the beginning of a code block. An illustration of the challenging tokens is shown in Figure 1. The remaining tokens, characterized by low loss values for LLMs, are referred to as confident tokens.

(2) In light of our findings, we propose Adaptive Temperature (AdapT) sampling, which dynamically adjusts the temperature coefficient. Compared to standard temperature sampling, AdapT sampling dynamically adjusts the temperature coefficient T according to the token type being sampled. Our motivation is that challenging tokens require LLM to explore diverse choices for the next token. For confident tokens, we should select tokens with high probabilities. Specifically, for challenging tokens, AdapT sampling utilizes a high T value to increase sample diversity. AdapT sampling uses a low T value for confident tokens to minimize randomness noise.

(3) Experimental results show that AdapT sampling can improve the pass@k metric on HumanEval and MBPP datasets with different sizes of LLMs. We apply the AdapT sampling to multiple LLMs with various sizes (from 2B to 13B) and conduct evaluations on two representative code generation datasets, i. e., HumanEval, and MBPP. Experimental results show AdapT sampling outperforms the SOTA decoding strategy. For example, it surpasses the pass@15 metric over the SOTA method by up to 13.6% on HumanEval. We further investigate the robustness of AdapT sampling to different hyperparameter settings and the quality of the code generated by AdapT sampling.

(4) Future directions. Based on our findings, we list the existing challenges and propose future research directions on developing effective decoding strategies for code generation.

Background

Code Generation with LLMs

LLMs are transformer-based models that are trained using large corpora of NL text and source code. In recent years, LLMs have achieved impressive results in automatic code generation. Among LLMs, the GPT family of LLMs from OpenAI is popular and powerful, including GPT-3 (175B parameters) (Brown et al. 2020), Codex (175B parameters) (Chen et al. 2021), etc. Since OpenAI LLMs are closed-source, there have been many attempts to reproduce similar LLMs, such as CodeGen (Nijkamp et al. 2022a), CodeGeeX (Zheng et al. 2023a), InCoder (Fried et al. 2022a).

Decoding Strategy

Given a requirement x, LLMs rely on a decoding strategy to auto-regressively generate the code. The decoding strategy determines how LLMs select the next token y_t based on the context $y_{<t}, x$. $y_{<t}$ is the token sequence that has been generated. There has been a series of works exploring decoding strategies for NL generation, and these methods have been subsequently applied to code generation. They can be classified into two categories: search-based and sampling-based methods.

Search-based Decoding Stratrgy Greedy search (Mou et al. 2015) is one of the most commonly used decoding strategies. In greedy search, the model selects the next token which maximizes the probability: $y_t = \arg\max_y p(y_t|y_{<t}, x)$. Despite its simplicity, it may lead to overly conservative results and a lack of diversity (See et al. 2019). Beam search (Freitag and Al-Onaizan 2017) is an improved version of greedy search. This algorithm retains top B (beam size) tokens with the highest probability. However, it has been found that beam search results in degenerations such as repetitions and empty (Holtzman et al. 2019).

Sampling-based Decoding Strategy The degenerations such as empty sequences and repetitions can be alleviated using sampling decoding, which randomly selects the next token based on predicted probability.

Temperature sampling (Ackley, Hinton, and Sejnowski 1985) has been applied widely, it uses a temperature coefficient T (usually $\in [0, 1]$) to control the size of the sampling randomness. Given a probability distribution p and temperature T, temperature sampling re-estimate the distribution as:

$$p'(y_t|y_{<t}, x) = \frac{\exp(\frac{\log(p(y_t|y_{<t},x))}{T})}{0.. \sum_{j=1}^{n} \exp(\frac{\log(p(y_j|y_{<t},x))}{T})} \quad (1)$$

In addition, researchers propose Top-k (Fan, Lewis, and Dauphin 2018) sampling to further improve performance. At each step, Top-k sampling filters the k most probable next tokens and redistributes the probability among these k tokens for sampling. However, the unreliable tail problem in the Top-k sampling may affect the sampling quality. Top-p sampling (Holtzman et al. 2019) eliminates the unreliable tail problem by rescaling the probability among the smallest token set whose cumulative probability reaches the threshold p.

LLMs usually use the method of combining temperature sampling and Top-p sampling to achieve SOTA results (Chen et al. 2021; Nijkamp et al. 2022a; Fried et al. 2022a). Specifically, the probability distribution is first rescaled with temperature T. After this, Top-p sampling is employed to derive the final results. Existing work (Chen et al. 2021) finds out that temperature coefficient T has an obvious influence on the code generation results. Increasing the T value can enhance the chance of exploring the correct answers, but this comes at the cost of introducing more errors in the generation results. Therefore there is a need to develop more effective sampling methods specifically for code generation.

Analysis of the code generation process

In this section, we first investigate the differences between NL generation with LLMs and code generation with LLMs. We compared the loss distributions (i. e., cross-entropy loss (Brownlee 2019)) of NL text to ones of source code.

Furthermore, we analyze the fluctuations of loss values of code tokens within code snippets. We find that code tokens

statistical evaluation of all 40 samples

metric	NL	PL
Standard Deviation	0.30	0.24
Skewness Coefficient	4.11	3.31
Perplexity	4.73	1.61

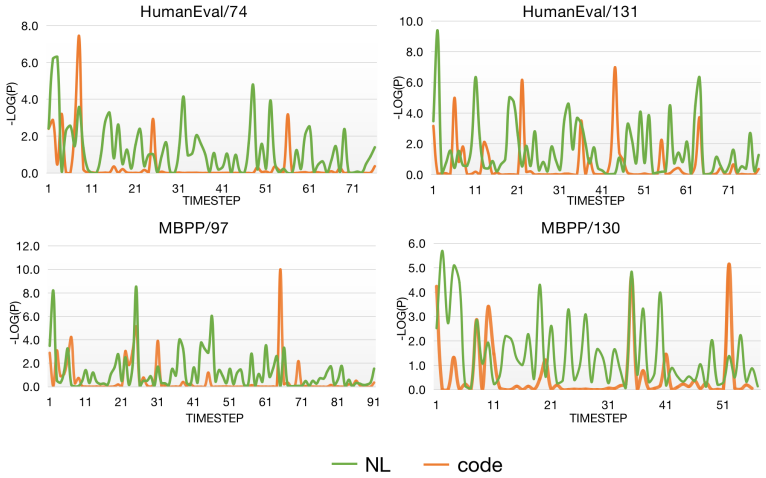

Figure 2: Comparison of loss distributions on NL text and source code. The left table shows the statistical results of different distributions on 40 samples. The right figures show several examples of loss distributions on NL text and source code.

can be categorized into challenging tokens and confident tokens. Based on the analysis, we discuss the challenges encountered in code generation.

Loss Distribution Comparison

In this section, we analyze the differences between the process of generating NL text and source code with LLMs. We choose loss distribution as the comparison metric.

We conduct experiments with a powerful LLM for source code - CodeGen. We select the CodeGen-mono with 2 million parameters as the base model. This model is trained with a 635GB code corpus and 1159GB English text data. Therefore, CodeGen has the ability to generate NL text and code. The datasets used in this section are HumanEval (Chen et al. 2021) and MBPP (Austin et al. 2021), which are representative datasets in code generation.

First, we randomly select 40 code samples from HumanEval and MBPP datasets. Then, we manually write NL descriptions for each code snippet, which describe the overall process of code. As a result, we obtain 40 NL descriptions aligned with their corresponding code snippets. Next, we gather the loss values of the CodeGen model on these NL descriptions and code snippets, respectively.

We use various metrics (e. g. mean value (Runnenburg 1978), standard deviation, skewness, and perplexity) to compare the loss distributions of NL descriptions and source code. Standard deviation (Bland and Altman 1996) reflects the average amount of variability. Skewness (Brown 2011) is a measure of the asymmetry of the probability distribution of a real-valued random variable about its mean. Perplexity (Brown et al. 1992) is a measurement of how confidently an LLM predicts a sample. As shown in the table in Figure 2, the average value of losses on NL descriptions is higher than the one on source code. When compared with code, NL suffers greater variation in prediction loss during generation. The value of the skewness shows that there are more tokens with large loss values in NL descriptions than in the source code. The LLM also has a higher perplexity for NL descriptions than for source code. We show a few examples in Figure 2 to visualize the differences.

These differences arise due to the fact that source code has a more strict syntax and semantics compared to NL. Therefore, when generating code, some tokens can be easily inferred based on grammatical rules, and LLMs can confidently generate these tokens with low loss values. In contrast, NL allows greater freedom in word usage and often presents multiple viable choices for the same context, which results in high loss values.

Additionally, we find that **a number of peaks occur in loss distributions of source code,** i. e. **tokens with a higher loss value than nearby tokens.** A higher loss value means that it is more difficult to make correct predictions at these locations. We call the tokens with peak loss values **challenging tokens**. As for the tokens that have low loss values, the model has more confidence in predicting them correctly. We refer to them as **confident tokens**.

In-depth Study of Code Tokens

This section provides a detailed investigation of the challenging tokens and confident tokens. We analyze samples from the MBPP (Austin et al. 2021) (500 samples), HumanEval (Chen et al. 2021) (164 samples), and APPS (Hendrycks et al. 2021) (train set: 5000 samples) datasets. We use CodeGen-2B to generate the ground truth code in these datasets and collect the corresponding loss values.

First, we define the predictive difficulty (PD) of a token, which is the rank (%) of the token loss among all token loss values in the code snippet, and compute PDs for all tokens. Then, we split each code snippet into code lines and integrate (average) the tokens' PDs in the same position of different code lines to obtain the position prediction difficulty. Finally, we statistically average the position prediction difficulty across all codes in the dataset. We omit positions where the cumulative data numbers of these positions are below 5% of the overall token numbers. Figure 3 shows the position prediction difficulty of different token positions. The results

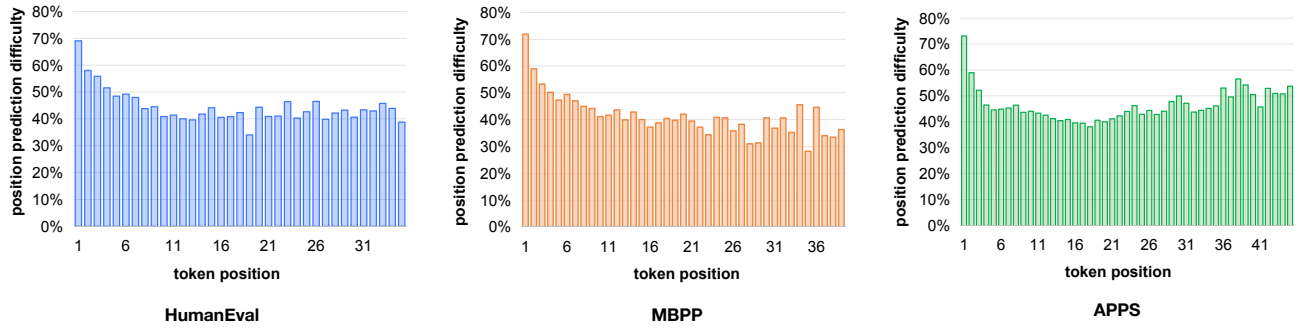

Figure 3: The prediction difficulty of different positions, the x-axis represents the position of the token within the line of code, and the y-axis represents the prediction difficulty of the token.

reveal the regularity that the tokens in the first position of code lines have the highest prediction difficulty of 69.0% (HumanEval), 73.1% (MBPP), and 71.9% (APPS). Therefore, we assume that the first position in each code line is where challenging tokens tend to appear and conduct further experiments.

We distinguish between challenging tokens and confident tokens based on PD. We introduce a threshold H to separate two types of tokens. Tokens with PD $> H$ are categorized as challenging tokens, while tokens with PD $< H$ are categorized as confident tokens. We set H=0.9, and the proportion of challenging tokens appearing in the first position was 24.8% (HumanEval), 22.6% (MBPP), and 28.1% (APPS), respectively. The results confirm our assumption that the challenging tokens are not randomly distributed for different locations, they tend to appear at the first position of each code line. We also vary H from 0.5 to 0.9 to investigate the distributions of challenging tokens and confident tokens. Results are shown in the appendix.

In order to further investigate the properties of challenging tokens, we distinguish the tokens at the first position of each line based on whether it is the initial token of a code block. A code block in Python is a piece of Python program text that can be executed as a unit. The code block begins at the first indented statement and continues until the indentation returns to a previous level. The initial tokens in each code block have a prediction difficulty of 0.798 (HumanEval), 0.834 (MBPP), and 0.825 (APPS) which is significantly higher than the first positions of other code lines. We derive that among the first positions of code lines, **the initial token of each code block is the most likely place for a challenging token to appear**.

This can be attributed to the fact that LLMs need to determine the next control structure after a block of code is presented, which increases prediction difficulty. LLMs can easily generate the next token after receiving the initial token since the strict syntax rules limit the scope of variations.

AdapT sampling

In light of our findings, we propose a simple yet effective decoding method, AdapT sampling (Adaptive Temperature Sampling), which adjusts the temperature coefficient T for different tokens. Specifically, for the challenging tokens, which LLMs struggle to predict correctly, AdapT sampling uses a high temperature coefficient which introduces more diverse tokens. On the other hand, for confident tokens, the temperature coefficient is set to a small value to minimize tailed noise. Specifically, we formulate AdapT sampling as follows:

$$p'_i = \frac{\exp(\frac{\log(p_i)}{T(t)})}{\sum_{j=1}^{n} \exp(\frac{\log(p_j)}{T(t)})} \tag{2}$$

$$T(t) = \begin{cases} a & \text{if } y_t \text{ is the code block initial token} \\ b & \text{else} \end{cases} \tag{3}$$

where T is the temperature coefficient and t is the sample timestep. $a, b \in [0, 1]$ $(a > b)$ are hyperparameters that control the degree of sampling randomness.

Experiments

Benchmarks

HumanEval (Chen et al. 2021) is a Python code generation benchmark with 164 test samples. Each sample consists of a manually written programming problem, which consists of a natural language requirement, a function signature, and several unit tests. It asks LLMs to generate the function body based on the requirement and the signature. The unit tests are used to check the correctness of generated functions.

MBPP (Austin et al. 2021) contains 974 programming problems collected from real-world communities. Solving these programming problems require simple numeric manipulations and the basic usage of standard libraries. Each problem contains an English requirement, a Python function signature, and three test cases. We take the requirement and the function signature as input and leverage LLMs to generate the function body. Then, the generated code is evaluated using test cases.

Base Models

In this paper, we select three representative open-source LLMs as base models: CodeGen, InCoder, and CodeGeeX.

Metric	pass@5	pass@10	pass@15
CodeGen:			
SP, T=0.2	29.2	31.3	32.3
SP, T=0.4	33.4	37.9	40.8
SP, T=0.6	33.1	38.0	40.8
SP, T=0.8	32.7	39.7	43.9
AdapT	**34.4**	**40.1**	**43.9**
InCoder:			
SP, T=0.2	22.8	25.9	27.4
SP, T=0.4	25.0	29.8	32.3
SP, T=0.6	24.5	30.0	32.9
SP, T=0.8	22.3	28.6	32.9
AdapT	**25.8**	**31.6**	**35.3**
CodeGeeX:			
SP, T=0.2	24.1	27.1	28.7
SP, T=0.4	27.0	30.5	32.3
SP, T=0.6	27.7	32.5	35.4
SP, T=0.8	27.1	33.0	36.0
AdapT	**29.4**	**36.3**	**40.9**

Table 1: The performance (pass@5, 10, 15) for CodeGen-2B, InCoder-6B, and CodeGeeX-13B on the HumanEval dataset using AdapT sampling and SOTA (SP) method.

Metric	pass@5	pass@10	pass@15
CodeGen:			
SP, T=0.2	33.1	36.5	38.4
SP, T=0.4	37.0	42.1	45.0
SP, T=0.6	37.1	43.5	47.0
SP, T=0.8	35.2	43.1	47.0
AdapT	**37.2**	**44.4**	**48.2**
InCoder:			
SP, T=0.2	23.0	27.4	29.2
SP, T=0.4	26.0	29.6	32.4
SP, T=0.6	24.6	30.4	34.6
SP, T=0.8	23.6	30.6	34.6
AdapT	**26.8**	**32.9**	**36.8**
CodeGeeX:			
SP, T=0.2	20.7	23.3	22.4
SP, T=0.4	23.7	28.4	31.0
SP, T=0.6	24.8	31.2	34.8
SP, T=0.8	25.4	31.2	35.6
AdapT	**25.8**	**32.2**	**36.0**

Table 2: The performance (pass@5, 10, 15) for CodeGen-2B, InCoder-6B, and CodeGeeX-13B on the MBPP dataset using AdapT sampling and SOTA (SP) method.

CodeGen (Nijkamp et al. 2022b) is a family of LLMs pre-trained on a large amount of NL texts and source code. It is trained with a 635GB code corpus and 1159GB English text data. In this paper, we select the CodeGen-mono with 2 million parameters as the base model.

InCoder (Fried et al. 2022b) is pre-trained with a large corpus of permissively licensed code (216GB). It can perform code generation and code infilling. In this paper, we use a version with 6.7 billion parameters, named InCoder-6B, for code generation.

CodeGeeX (Zheng et al. 2023b) is a multilingual LLM with 13 billion parameters. It is pre-trained on a large corpus of more than 20 programming languages and achieves impressive performance on code generation.

Despite OpenAI models' impressive performance on code generation, we can not access the probability distribution with only API calls. Therefore, we ignore these models in this paper.

Baselines

In this paper, we choose the SOTA approach mentioned in the background section as the baseline. For the sake of brevity, we refer to the baseline (the combination of temperature and Top-p sampling) as SP in the following sections.

Implementation Details

We run all of our experiments on NVIDIA V100 GPU with 32GB memory. For our experimental datasets, the maximum generated length is 500. All experiments are conducted in a zero-shot setting which means we directly feed the input requirement into LLMs without examples. Then we extract generated programs from the model's output.

Metric

Pass@k Pass@k (Chen et al. 2021) measures the functional correctness of the generated code by executing test cases. We use the unbiased version of pass@k, where $n \geq k$ samples are generated for each problem, and $c \leq n$ is the number of correct samples that pass test cases. Pass@k is calculated as follows:

$$\text{pass@}k = \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \tag{4}$$

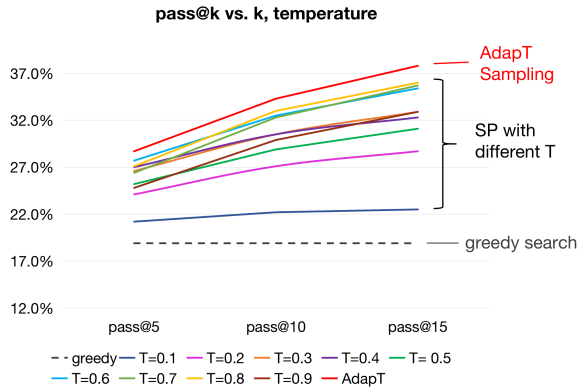

Figure 4: The performance of CodeGeeX-13B on the HumanEval dataset with dense temperature settings.

Main Results

We apply AdapT sampling to three base models on two code generation datasets. The performance is shown in Ta-

Dataset	HumanEval	MBPP
CodeGen:		
Greedy Search	23.8	24.0
SP-best	22.2	24.6
AdapT	23.3	24.9
InCoder:		
Greedy Search	14.0	12.3
SP-best	15.3	13.8
AdapT	15.2	14.8
CodeGeeX:		
Greedy Search	18.9	13.6
SP-best	18.0	13.4
AdapT	18.8	13.5

Table 3: Pass@1 results for CodeGen-2B, InCoder-6B, and CodeGeeX-13B using different decoding strategies on HumanEval and MBPP datasets.

ble 1 and Table 2. Following the previous works (Nijkamp et al. 2022a; Zheng et al. 2023a; Fried et al. 2022a), we set p=0.95 for all our experiments. Following the previous work (Chen et al. 2021), we set the values of temperature of baseline as 0.2, 0.4, 0.6, and 0.8 respectively. We experimentally select the values of a and b, the settings are present in the appendix. The sampling number n is 15. As shown in Table 1 and Table 2, AdapT sampling outperforms the SOTA method in terms of pass@5, pass@10, and pass@15 on HumanEval and MBPP datasets. Pass@15 represents the number of problems solved since the sampling number n is 15. AdapT sampling with the highest pass@15 indicates that it solves the most problems. Notably, on the HumanEval dataset, AdapT sampling can enhance the pass@15 of CodeGeeX from 36.0% to 40.9%, reaching a 13.6% improvement. Meanwhile, the number of solved problems increased from 60 to 67. When run with a constant value of T, increasing T will improve the number of problems solved, but drastically reduce pass@5 (represents the proportion of correct answers sampled for each question) due to the introduction of randomness (Chen et al. 2021; Holtzman et al. 2019). AdapT sampling can dynamically adjust the sampling randomness, thus minimizing the noise associated with increasing T, hence consistently demonstrating an improvement in pass@5, 10, and 15.

Analysis and Discussion

We conduct an in-depth and comprehensive analysis of AdapT sampling's capabilities.

pass@1 The pass@1 metric represents the probability that, among multiple pieces of generated code, the first piece chosen would successfully pass a given test case. Note that the pass@1 are very strict metrics and are hard to be improved. We compare the pass@1 results of AdapT sampling with greedy search (which usually has a high pass@1 value) and the best pass@1 results of SOTA (SP-best) and show the results in Table 3. Our method outperforms SP-best in 83.3% cases on the pass@1 metric. Meanwhile, AdapT sampling can reach a comparable pass@1 when compared with greedy search. Greedy search can only sample one answer per question, whereas our method can sample n answers and increase the number of solved questions.

Compare with Different T Settings In order to explore the upper bound performance of SP, we set the temperature value in SP more densely from 0 to 1, taking a value every 0.1 intervals. The results are shown in Figure 4. The performance is not displayed on the resulting graph when $T > 1$, since it drops significantly when $T > 1$. This figure shows the performance of CodeGeeX-13B on the HumanEval dataset. The results of other models can be found in the appendix. AdapT sampling significantly outperforms temperature sampling at all settings on pass@5, pass@10, and pass@15 on all evaluated models. A model can only answer 31 questions correctly with greedy search. Using the AdapT sampling method, LLM can solve twice as many problems (62) as greedy search.

Hyperparameters Analysis There are two hyperparameters involved in AdapT sampling: a and b. In this section, we examine how changes to these two parameters affect the code generation results. The model and dataset used in this section are CodeGeeX-13B and HumanEval. The result of the MBPP dataset is presented in the appendix.

First, we fix $a = 0.8$, and we take a b value every 0.1 step from 0.1 to 0.9, the experimental results are displayed in the upper part of Figure 5. AdapT sampling outperforms SP ($T = 0.8$) on pass@1, pass@5, pass@10, and pass@15 metrics when b equals 0.3, 0.5, 0.6, and 0.7.

Then we set $b = 0.3$, and vary the value of a from 0.1 to 0.9 with a step of 0.1. The results are shown at the bottom half of Figure 5. It can be seen from the experimental results that when $a > b$, the results of pass@5, pass@10, and pass@15 can be effectively improved. Additionally, changing a from 0.2 to 0.9 can increase pass@15 from 32.9% to 37.8%, achieving a 14.9% improvement.

The results indicate that AdapT sampling can outperform SP under a variety of settings, which confirms the robustness of AdapT sampling over hyperparameters.

Code Quality Evaluation In this section, we analyze the quality of code generated by different sampling methods on the HumanEval dataset. We collect the execution results of the generated code of three models. We select the best-performed baselines and compare them with AdapT sampling. The execution result is presented in Figure 6. AdapT sampling can generate more correct samples (passed) than baselines on all models. Using AdapT sampling, CodeGen-2B, InCoder-6B, and CodeGeeX-13B can generate 517, 333, 382 correct codes, which outperforms SP up to 17.2%, 8.8%, 14.0% higher than sampling with the SP method, respectively.

There are fewer TypeError and SyntaxError in the code generated by the three models using AdapT sampling than SP. By using AdapT sampling in CodeGen and CodeGeeX models, the occurrence of NameError can be reduced by 44.3% and 23.4%. In CodeGen and CodeGeeX, AdapT sampling reduces the incidence of NameError by 44.3% and 23.4%. The Else section in Figure 6 contains Indentation-

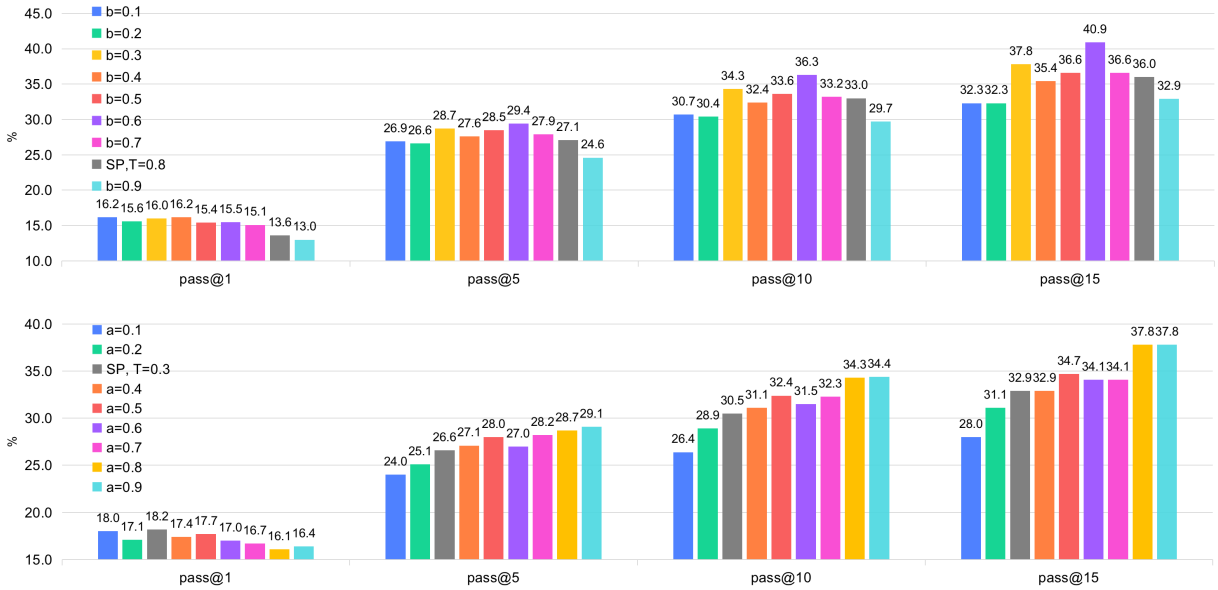

Figure 5: Quantitative analysis of the two hyperparameters (a, b) in AdapT sampling. The upper half shows the results vary b from 0.1 to 0.9 when fixing $a = 0.8$. The bottom half shows the results of fixing b = 0.3 and taking values of a from 0.1 to 0.9. When $a > b$, the AdapT sampling can outperform SP with multiple settings.

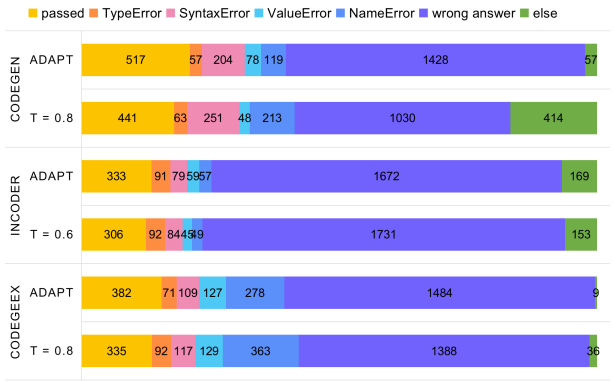

Figure 6: Evaluation of the generated code quality of AdapT sampling. Compared with SP, AdapT sampling is able to reduce syntax errors and increase correct samples.

Errors, RecursionErrors, UnboundLocalErrors, RuntimeError, etc. These types of errors rarely occur in our AdapT sampling. The reason for this is that AdapT sampling uses a smaller T inside the code block, which reduces noise from randomness, improves the coherence of the sampled code, and therefore reduces syntax errors. For AdapT sampling, the most common error type is the wrong answer, showing that our method suffers from incorrect code logic. Taking steps to solve this issue can be a great improvement for our method in the future.

Future Work

In this section, we discuss the remaining challenges of decoding strategies in code generation and provide some possible directions to facilitate other researchers.

- This paper is the first step toward the decoding strategy for code generation. We obtain observations based on extensive statistical analysis and further improve existing decoding strategies. In the future, we can use an individual neural network to learn how to dynamically tune T to improve the accuracy of code generation.
- In practice, software development often relies on specific domain knowledge, such as private code libraries and code specifications. Existing decoding strategies ignore these issues. In the future, we can introduce domain knowledge to constrain the decoding process, improving the usability of code generation systems in real-world scenarios.
- As shown in Figure 6, AdapT sampling reduces the occurrence of syntax errors but contains many wrong answer errors. It shows that LLMs are suffering from incorrect code logic, and generating code from scratch is very challenging. In the future, we can design a multi-stage decoding strategy, which steers LLMs to generate code progressively. For example, LLMs first generate a natural language plan, then implement an executable program based on the plan.

Conclusion

This paper is the first step toward the decoding strategy for code generation. We statistically analyze the loss distribution of source code and find out that code tokens can be categorized into challenging tokens and confident tokens. Moreover, challenging tokens often appear in the initial of a code block. Based on the insights, we propose AdapT sampling which dynamically adjusts the temperature coefficient through sampling and has proven its effectiveness on code

generation datasets. Finally, we present several challenges and insights in developing a more advanced decoding strategy for code generation and we look forward to further exploring its potential in future research.

References

Ackley, D. H.; Hinton, G. E.; and Sejnowski, T. J. 1985. A learning algorithm for Boltzmann machines. *Cognitive science*, 9(1): 147–169.

Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. 2021. Program Synthesis with Large Language Models.

Black; and E, P. 2012. greedy algorithm, Dictionary of Algorithms and Data Structures. *US Nat. Inst. Std. & Tech Report*, 88: 95.

Bland, J. M.; and Altman, D. G. 1996. Measurement error. *BMJ: British medical journal*, 312(7047): 1654.

Brown, P. F.; Della Pietra, S. A.; Della Pietra, V. J.; Lai, J. C.; and Mercer, R. L. 1992. An estimate of an upper bound for the entropy of English. *Computational Linguistics*, 18(1): 31–40.

Brown, S. 2011. Measures of shape: Skewness and kurtosis.

Brown, T. B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D. M.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. *CoRR*, abs/2005.14165.

Brownlee, J. 2019. *Probability for machine learning: Discover how to harness uncertainty with Python*. Machine Learning Mastery.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Ponde, H.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; Ray, A.; Puri, R.; Krueger, G.; Petrov, M.; Khlaaf, H.; Sastry, G.; Mishkin, P.; Chan, B.; Gray, S.; Ryder, N.; Pavlov, M.; Power, A.; Kaiser, L.; Bavarian, M.; Winter, C.; Tillet, P.; Such, F. P.; Cummings, D. W.; Plappert, M.; Chantzis, F.; Barnes, E.; Herbert-Voss, A.; Guss, W. H.; Nichol, A.; Babuschkin, I.; Balaji, S. A.; Jain, S.; Carr, A.; Leike, J.; Achiam, J.; Misra, V.; Morikawa, E.; Radford, A.; Knight, M. M.; Brundage, M.; Murati, M.; Mayer, K.; Welinder, P.; McGrew, B.; Amodei, D.; McCandlish, S.; Sutskever, I.; and Zaremba, W. 2021. Evaluating Large Language Models Trained on Code.

Fan, A.; Lewis, M.; and Dauphin, Y. 2018. Hierarchical Neural Story Generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 889–898. Melbourne, Australia: Association for Computational Linguistics.

Freitag, M.; and Al-Onaizan, Y. 2017. Beam search strategies for neural machine translation.

Fried, D.; Aghajanyan, A.; Lin, J.; Wang, S.; Wallace, E.; Shi, F.; Zhong, R.; Yih, S.; Zettlemoyer, L.; and Lewis, M. 2022a. InCoder: A Generative Model for Code Infilling and Synthesis. In *The Eleventh International Conference on Learning Representations*.

Fried, D.; Aghajanyan, A.; Lin, J.; Wang, S.; Wallace, E.; Shi, F.; Zhong, R.; Yih, W.-t.; Zettlemoyer, L.; and Lewis, M. 2022b. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*.

Hendrycks, D.; Basart, S.; Kadavath, S.; Mazeika, M.; Arora, A.; Guo, E.; Burns, C.; Puranik, S.; He, H.; Song, D.; et al. 2021. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*.

Holtzman, A.; Buys, J.; Du, L.; Forbes, M.; and Choi, Y. 2019. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.; et al. 2022. Competition-level code generation with alphacode. *Science*, 378(6624): 1092–1097.

Mou, L.; Men, R.; Li, G.; Zhang, L.; and Jin, Z. 2015. On End-to-End Program Generation from User Intention by Deep Neural Networks. *ArXiv*, abs/1510.07211.

Nijkamp, E.; Pang, B.; Hayashi, H.; Tu, L.; Wang, H.; Zhou, Y.; Savarese, S.; and Xiong, C. 2022a. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*.

Nijkamp, E.; Pang, B.; Hayashi, H.; Tu, L.; Wang, H.; Zhou, Y.; Savarese, S.; and Xiong, C. 2022b. A conversational paradigm for program synthesis. *arXiv e-prints*, arXiv–2203.

Runnenburg, J. T. 1978. Mean, median, mode. *Statistica Neerlandica*, 32(2): 73–79.

See, A.; Pappu, A.; Saxena, R.; Yerukola, A.; and Manning, C. D. 2019. Do Massively Pretrained Language Models Make Better Storytellers? In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*, 843–861. Hong Kong, China: Association for Computational Linguistics.

Zhang, X.; Sun, M.; Liu, J.; and Li, X. 2021. Improving diversity of neural text generation via inverse probability weighting.

Zheng, Q.; Xia, X.; Zou, X.; Dong, Y.; Wang, S.; Xue, Y.; Wang, Z.; Shen, L.; Wang, A.; Li, Y.; Su, T.; Yang, Z.; and Tang, J. 2023a. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Evaluations on HumanEval-X. arXiv:2303.17568.

Zheng, Q.; Xia, X.; Zou, X.; Dong, Y.; Wang, S.; Xue, Y.; Wang, Z.; Shen, L.; Wang, A.; Li, Y.; et al. 2023b. Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x. *arXiv preprint arXiv:2303.17568*.

Appendix

Hyperparameter Settings

Table 4: Hyperparameters used in Main Result Section

Dataset	HumanEval		MBPP	
Model	a	b	a	b
CodeGen	0.8	0.4	0.7	0.6
InCoder	0.8	0.2	0.7	0.4
CodeGeeX	0.8	0.6	0.6	0.5

Table 5: Hyperparameters used in Pass@1 Section

Dataset	HumanEval		MBPP	
Model	a	b	a	b
CodeGen	0.2	0.01	0.2	0.1
InCoder	0.3	0.01	0.2	0.01
CodeGeeX	0.1	0.01	0.2	0.01

Analysis of challenging tokens

We change H from 0.5 to 0.9 and observe the occurrence of challenging tokens appearing in the first position of code lines, and the results of three datasets (i. e. HumanEval, MBPP, APPS) are shown in Figure 7.

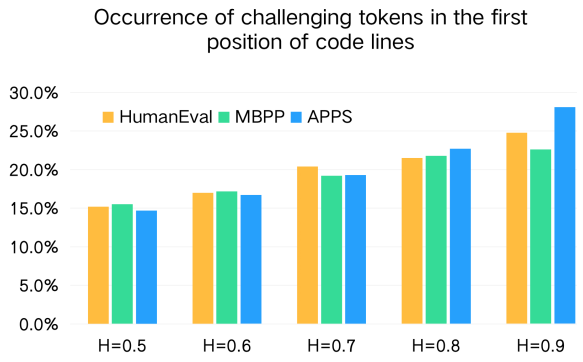

Figure 7: The performance of CodeGeeX-13B on the MBPP with different T.

The results show that as the H increases, the chances of large loss tokens appearing at the beginning of the code line increase. This indicates that challenging tokens are more likely to appear at the beginning of the code lines.

Comparison with different T

For brevity, we skip the data already shown in Table 1 and Table 2 (i. e. T = 0.2, 0.4, 0.6, 0.8). Since greedy search can only generate one result, all the results of the greedy search are its pass@1 result in the following figures.

Figure 8 shows the performance of CodeGeeX-13B on the MBPP with different T. Figure 9 and 10 show the performance of CodeGen-2B on the HumanEval and MBPP datasets with different T. Figure 11 and 12 show the performance of InCoder-6B on the HumanEval and MBPP datasets with different T.

The results show that AdapT sampling outperforms the SOTA method in all T settings on all models. This indicates that AdapT sampling can exceed the upper bound of the SOTA method.

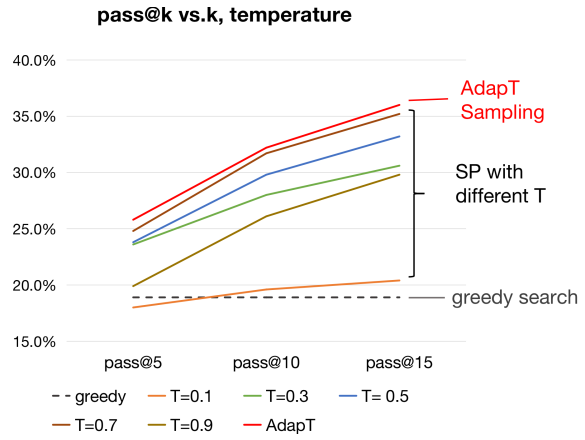

Figure 8: The performance of CodeGeeX-13B on the MBPP with different T.

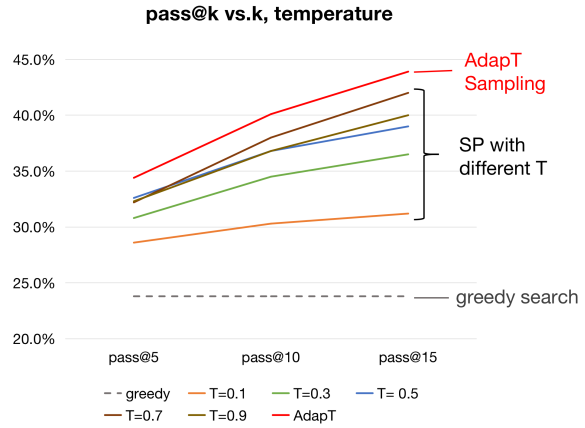

Figure 9: The performance of CodeGen-2B on the HumanEval with different T.

Hyperparameters Analysis

The hyperparameters analysis of the CodeGen-2B model on HumanEval is shown in Figure 13.

Case Study

We show an example of code generation with AdapT sampling and SOTA baseline in Figure 14. It can be seen that the SOTA method fails to generate the correct answer under different T settings, while AdapT sampling generates the code that passes all the test cases and with the correct code logic.

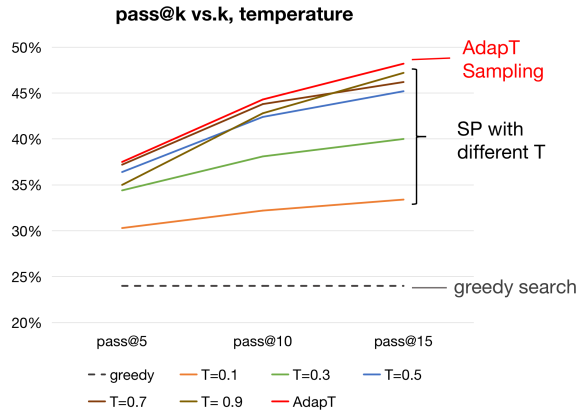

Figure 10: The performance of CodeGen-2B on the MBPP with different T.

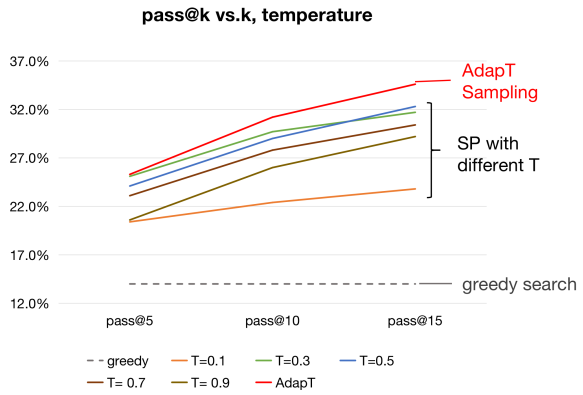

Figure 11: The performance of InCoder-6B on the HumanEval with different T.

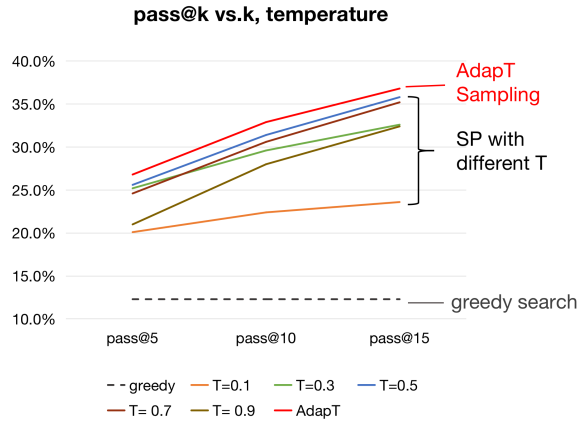

Figure 12: The performance of InCoder-6B on the MBPP with different T.

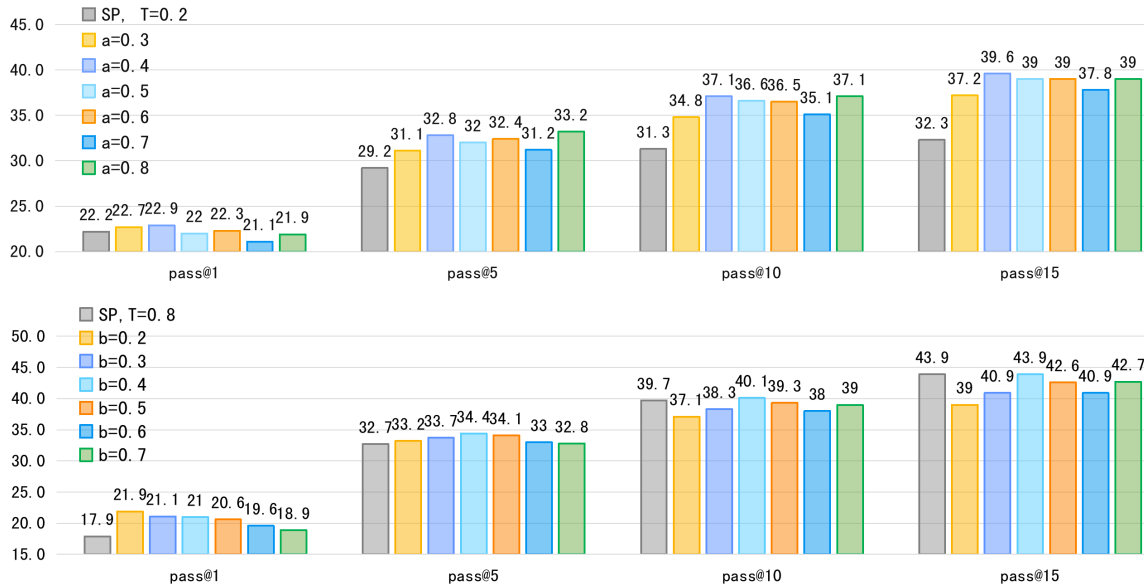

Figure 13: The hyperparameters analysis of CodeGen-2B model on HumanEval dataset.

Intent (HumanEval/106):

```
def f(n):
    """ Implement the function f that takes n as a parameter,
    and returns a list of size n, such that the value of the element at index i is the factorial of i if i is even or the sum of numbers from 1 to i otherwise.
    i starts from 1.
    the factorial of i is the multiplication of the numbers from 1 to i (1 * 2 * ... * i).
    Example:
    f(5) == [1, 2, 6, 24, 15]
    """
```

SP, T=0.8:

```
list1 = []
for i in range(1, n + 1):
  if i % 2 == 0:
    list1.append(i * i)
  else:
    list1.append(sum(range(1, i + 1)))
return list1
```

SP, T=0.2:

```
fact = 1
lst = []
for i in range(1, n+1):
  if i % 2 == 0:
    fact = fact * i
  else:
    fact = fact + i
    lst.append(fact)
return lst
```

Ground Truth:

```
ret = []
for i in range(1,n+1):
  if i%2 == 0:
    x = 1
    for j in range(1,i+1): x *= j
    ret += [x]
  else:
    x = 0
    for j in range(1,i+1): x += j
    ret += [x]
return ret
```

SP, T=0.4:

```
factorial = []
for i in range(1,n+1):
  if i%2 == 0:
    factorial.append(i)
  else:
    factorial.append(i*(i-1))
return factorial
```

SP, T=0.6:

```
fac = [1]
for i in range(2, n+1):
  if i%2 == 0:
    fac.append(fac[-1]*i)
  else:
    fac.append(fac[-1]+i)
return fac
```

AdapT:

```
import math
l = []
for i in range(1, n+1):
  if i % 2 == 0:
    l.append(math.factorial(i))
  else:
    l.append(sum(range(1, i+1)))
return l
```

Figure 14: An example of generating code with AdapT sampling and SOTA method. Only AdapT sampling generates the correct answer.